3、消息列表卡顿优化 —— 开发中解决过最难的问题是什么？

遇到的问题：

用户反馈语音房存在卡顿严重的问题。

原因

业务实际分析：

- 消息类型多：文本、图片、富文本、礼物、红包、提示消息
- 消息结构复杂：复杂的适配问题、图层堆叠，且大部分都带有气泡
- 消息里有一些法消息人的标识icon
- 稍大的房间会存在一瞬间刷很多消息的情况

场景复现与检测

线下检测

- QA模拟复现，用压力数据回放：把线上真实消息按时间序列回放到模拟器/测试机（包含富文本、图片、气泡、icon），并在不同设备上做压力测试。
  - 复现突发场景：模拟“同一时刻 N 条消息到达”并记录 FPS、主线程 CPU。
- 使用 Instruments 的 Time Profiler / Core Animation / Allocations 同步对比。

工具	主要目的	检测什么卡顿？	最适合
Time Profiler	看 CPU	“为什么这一帧做得太慢？”	你的代码太重、布局太慢、decod
Core Animation	看 GPU / 渲染	“为什么掉帧？”	过度绘制、离屏渲染、图层太复杂
Allocations	看内存 & 对象创建	“为什么 scrolling 卡顿抖动？”	频繁创建对象、cell 配置太重、图

我一般会同时打开 **Time Profiler + Core Animation + Allocations** 三个工具：

如果有异常，会有峰值出现。使用mark 去界定“问题发生点前后”的差异。

- **Time Profiler** 盯 CPU，找出 layout 和富文本的热点函数。

    - CPU usage 峰值暴涨
  - **Core Animation** 看是否存在离屏渲染或大图导致 GPU 压力
    - 帧率下降（FPS 掉到 20 以下）
  - **Allocations** 观察 Cell 是否创建太多对象、是否缺少缓存
    - 对象创建峰值、内存暴涨

线上检测

iOS性能优化实践01： 卡顿

- FPS线上卡顿检测工具
  - 基于CADisplayLin做帧率回调次数计算，CADisplayLink是Core Animation提供的一个类似NSTimer的定时器，它会在屏幕每次刷新回调一次，所以它也是以runloop的帧率为标准。所以只需统计每秒方法执行的次数，次数/时间就可以得出帧率了。但是它无法真正定位到性能
- 卡顿检测工具2： runloop+信号量的方式：
  - 根据RunLoop的运行周期，在RunLoop进入睡眠之前的KCFRunLoopBeforeSources和唤醒之后KCFRunLoopAfterWaiting这两个状态是RunLoop处理主线程消息的主要状态，如果在这两个状态停滞的时间过长的话，那么可以认为是线程受阻了。
  - 具体方案：开一个单独的线程监听主线程的runloop状态，同时以while循环不断查看主现场的当前状态，并及时。这里有个细节，不能让while一直循环，需要让信号量来控制while循环是否继续执行，主要就是用self.dispatchSemaphore.wait来懒觉，他可以在收到self.dispatchSemaphore.signal()继续往后执行，也可以在没收到信号量，但是达到了设置的时间阈值时继续往后执行。这样就可以知道在某一个状态变更或者超时了多少时间。然后再把卡顿时间丢出去。

代码块

```
// 监控卡顿的循环方法
    private func hangsLoop() {
        // 等待信号量，超时时间为hangsInterval毫秒
        let result = self.dispatchSemaphore.wait(timeout: DispatchTime.now() + .milliseconds(self.hangsInterval))
        // 获取当前的绝对时间
        let current = CFAbsoluteTimeGetCurrent()
        // 如果等待超时
        if result == .timedOut {
            // 如果runloop的活动状态是afterWaiting或deforeSource
            // 说明发生了卡顿
            if self.runloopActivity == .afterWaiting || self.runloopActivity == .beforeSources {
                // 卡顿次数 +1
```

```
                    self.timeoutCount += 1
                    // 计算持续卡顿时间
                    let cost = current - hangeTagTime
                    // 每发生一次卡顿就回调出去
                    self.delegate?.hangsHitted(cost: cost, count: timeoutCount)
                } else {
                    // do nothing
                }
            } else {
                // 如果信号量被触发（没有超时）
                // 重置超时计数
                self.timeoutCount = 0
                // 更新上一次检测时间
                hangeTagTime = current
            }
        }

        // 主线程RunLoop的回调函数
        private func observerCallBack() -> CFRunLoopObserverCallBack {
            return { (_, activity, context) in
                // 从context中获取到当前对象
                let weakSekf = Unmanaged<HangsMonitorThread>.fromOpaque(context!).takeUnretainedValue()
                // 将主线程RunLoop的activity赋值给当前线程的runloopActivity
                weakSekf.runloopActivity = activity
                // 发送信号，唤醒等待的线程。
                weakSekf.dispatchSemaphore.signal()
            }
        }
```

优化思路一页总结：

UI层的具体优化：（参考了很多MessageKit的方案）

> **通过“超细粒度重用 + 稳定且增量的数据源更新”，减少 UI 重建、减少约束计算、减少对象分配，从根本上提升大房间消息流的吞吐能力。**
>
> MessageKit思想：
>
> ✔ **布局计算与视图展示解耦**
>
> ✔ **富文本提前渲染，不在 cell 中拼装**
>
> ✔ **多类型消息分类型布局**
>
> ✔ **所有 layout 都可缓存**

- 将自动适配改成手动布局：因为AutoLayout在高更新频率下会频繁触发 `layoutSubviews` 和文本测量
  - 技巧：position 和size分开来计算
- 富文本问题：子线程生成 TextAttachment 与 NSAttributedString / TextKit 或用CTFramesetter计算高度并缓存：
  - 技巧1：最后就将消息放到显示队列时进行子线程处理。
  - 技巧2: 使用 `NSTextStorage + NSLayoutManager + NSTextContainer` 在后台计算 `NSLayoutManager` 布局。
    - MessageKit 在后台创建一个 TextKit 排版系统（TextStorage + LayoutManager + TextContainer），预先计算文本消息的排版（glyph layout），得到最终文本的 boundingRect，从而提前知道 cell 高度和每行的布局信息。

    > MessageKit 采用 TextKit 的原因：
    > - 可后台排版（减轻主线程压力）
    > - 更准确的文本高度
    > - 更好的富文本布局（尤其 emoji + 附件）
    > - 对动态内容（mention、链接、highlight）支持更强
  - 技巧3:
- 图片优化：？？？
  - 使用缩略图，头像、icon：提前降低采样率 downsample（服务器或客户端在 decode 前缩放）
  - 礼物图等动画图：使用缓存的缩略图，避免每次 decode 原图。SDWebImage 的 `downsample` / `context` 可以配置。
  - 禁止主线程解码：解码放后台并把 `CGImage` 或 `UIImage` 传回主线程。

> SDwebImage都有，只需进行对应配置即可
>
> `.scaleDownLargeImages` → SDWebImage 内部使用 `CGImageSourceCreateThumbnailAtIndex` 自动 downsample
>
> `.avoidAutoSetImage` → 避免解码在主线程完成，手动设置 image
>
> `imageThumbnailPixelSize` → 确保生成的图片尺寸接近实际显示尺寸，节省内存
>
> 背景线程 decode → 滑动不卡顿

- 气泡背景与圆角
  - 气泡通常重复使用，可以把气泡做成 layer-backed（`resizable image` 或 `stretchable`），避免每条消息 draw。对静态复杂气泡（带渐变或阴影）：

        - 在首次构造时把其 `CALayer` rasterize 成一张 bitmap（`shouldRasterize = true`）并缓存（注意 scale）
        - 或者使用 9-patch / `resizableImage(withCapInsets:)` 的方式避免重绘

    注意权衡： `shouldRasterize` 在分辨率变化或动画中可能带来反效果（invalidate cost），只在静态场景用。

    - 气泡使用 resizable image / layer-backed 方案；对静态气泡做 rasterize cache。

    > Resizable Image（可拉伸图片）
    >
    > **layer-backed图层缓存方案**
    >
    > - 将气泡 layer 设置 `shouldRasterize = true` // `Rasterize` 栅格化
    > - 先把 layer 渲染成一张 bitmap（离屏缓存）
    > - 后续直接用这张 bitmap 渲染，不用每次重绘

- 整体消息列表：
    - 使用 `UICollectionView` + `DiffableDataSource`
        - 我们使用 DiffableDataSource 的核心原因是它保证了数据 + UI 的一致性，并且可以通过 Snapshot Incremental 策略做到高吞吐量更新。
        - Diffable 的底层是基于稳定 ID 的最小编辑距离 Diff 算法，它会对旧 Snapshot 和新 Snapshot 做 O(n) 的差异计算，并自动生成 insert/delete/move/reload 的最小指令集。
            - Snapshot 是一个不可变的 UI 结构描述，它记录了当前列表的 Section 顺序和每个 Section 内 Item 的顺序。当我们构建新的 Snapshot 并 apply 时，Diffable 会内部用 C++ 的 Myers Diff 算法计算 old/new Snapshot 的差异，然后自动生成最小化 UI 操作并批量执行。
        - 在直播间这种消息瞬时爆量的场景下，我们不做全量快照 apply，而是从 currentSnapshot 取出最新快照，仅 append 新消息，这样系统只对底部几十条执行 diff，不会触发整表刷新，也不会引发布局地震

消息量：

- 消息量大：
    - 限流： 限制最大消息数 200 条展示消息。
- 双队列策略：
    - 我们实现了一个消息流控系统，就像给水管装了个水龙头。直播间消息太快会卡UI，所以我们先把消息存到缓冲队列，然后用定时器每0.5秒派发5条，确保UI能平稳渲染。还支持三种模式：正常慢速播放、用户点击快进、后台暂停只统计数量。这样既保证了流畅度，又不会丢消息。

测试：

- AB测试
- monkey测试

效果：

基本FPS都能达到50以上

底层原理参考：

iOS性能优化实践01： 卡顿